GateGuard Project Repository

github.com/Pushkar111/GateGaurd-IITGN-Databases-CS432

Assignment 3 Demonstration VIEW

Indian Institute of Technology Gandhinagar

Department of Computer Science & Engineering

CS 432 → Databases

Track-1 · Assignment 3 · Module A & B

GateGuard

ACID Transactions, WAL Crash Recovery
& Concurrent Stress Testing

Custom B+ Tree Engine • Write-Ahead Log • SELECT FOR UPDATE

Team Name: *Lumos*

Team Members

Name	Roll Number
Pushkar Nikunjkumar Modi	25120035
Chintan Bhikhabhai Akhaja	25120027
Krina Paghadar	24110169
Divisht	24110112
Akshata Pawar	25120026

Instructor: Prof. Yogesh K. Meena

Semester: II (2025–2026) **Deadline:** 6:00 PM, 5 April 2026

Contents

1 Introduction **2**
1.1 Context and Motivation 2
1.2 Scope and Requirements Coverage 2
1.3 The Three Relation Database 3
2 Module A: Transaction Engine **4**
2.1 System Architecture 4
2.1.1 Database Initialization (Cell 1) 4
2.2 Write-Ahead Log (WAL) 5
2.2.1 WAL Pipeline 5
2.2.2 WAL Implementation 5
2.2.3 Actual WAL Log Output 5
2.3 Atomicity 6
2.3.1 Atomicity Test: Cell 2 Output 7
2.4 Durability and Crash Recovery 7
2.4.1 Recovery Algorithm 7
2.4.2 Durability Test: Cell 3 Output 8
2.5 Multi-Table Atomicity: The 3-Relation Proof 9
2.6 Consistency Verification 10
2.7 ACID Summary 11
3 Module B: Concurrency and Stress Testing **12**
3.1 The Race Condition Problem 12
3.1.1 Visual Comparison: Before and After the Fix 12
3.2 The Fix 12
3.3 Race Condition Test Results 13
3.4 Failure Simulation Test Results 14
3.5 Locust Load Testing 15
4 Git Repository and Branch Strategy **17**
5 Observations and Limitations **18**
5.1 What Worked Well 18
5.2 Limitations 18
6 Conclusion **18**
7 Team Contributions **19**
A Module A File Structure **20**
B Module B Stress Test File Structure **20**
C Raw Test Data **20**

1. Introduction

1.1 Context and Motivation

GateGuard is our campus gate management system for CS432. We built it over three assignments. By the end of Assignment 2, it worked well for regular use. Guards could record entries and exits, role-based access control protected sensitive actions, and indexing kept queries fast.

But we had not solved failure handling yet. One important question was still open: what happens if the backend crashes in the middle of a write?

For example, if the system writes to Member(User) and crashes before writing to PersonVisit, the database can be left in a partial and inconsistent state. Without a transaction layer, there is no safe way to undo that half-complete operation.

Assignment 3 asked us to fix this in two modules. In Module A, we built a transaction engine with Write-Ahead Logging (WAL) and crash recovery on top of our custom B+ Tree database from Assignment 2. In Module B, we tested the live REST API under concurrent load, fixed race conditions that could create two active visits for one member, and ran realistic load tests with Locust.

1.2 Scope and Requirements Coverage

Table 1: Assignment 3 Requirements and Evidence

Requirement	Our Solution	Evidence	Status
At least 3 relations per transaction	Member + PersonVisit + GateOccupancy	Notebook Cells 3 & 4	✓ **PASS**
B+ Tree as sole storage engine	All 10 tables via order-4 BPlusTree	Notebook Cell 1	✓ **PASS**
Atomicity: all-or-nothing	LIFO Undo Stack rollback	Notebook Cell 2	✓ **PASS**
Durability: survive restart	WAL REDO recovery	Notebook Cell 3	✓ **PASS**
Consistency: B+ Tree invariant	ConsistencyChecker scan	Notebook Cell 5	✓ **PASS**
Isolation: concurrent safety	SELECT FOR UPDATE lock	Race test: 1/50	✓ **PASS**
4 failure scenarios tested	`failure_simulation_test.py`	`failure_results.json`	✓ **PASS**
Race condition stress test	`race_condition_test.py`	`race_results.json`	✓ **PASS**
Load testing under 50 users	Locust with weighted tasks	28–30 RPS sustained	✓ **PASS**

1.3 The Three Relation Database

Three tables form the core of every multi-relation transaction in Module A:

Table 2: Relations Used in ACID Transactions

Table	Primary Key	Key Columns	Role
Member	MemberID	Name, Email, Department, TypeID	Person identity
PersonVisit	VisitID	MemberID (FK), EntryGateID, ExitGateID, EntryTime, ExitTime	Entry/exit event
GateOccupancy	OccupancyID	GateID (FK), Timestamp, PersonCount, VehicleCount	Gate load snapshot

Storage note: All three tables live in separate B+ Trees with order=4. There is no PostgreSQL backing store for Module A. The B+ Tree IS the database.

2. Module A: Transaction Engine

2.1 System Architecture

The transaction engine sits on top of the `DatabaseManager` and `BPlusTree` we built in Assignment 2. Four Python modules handle all the hard work:

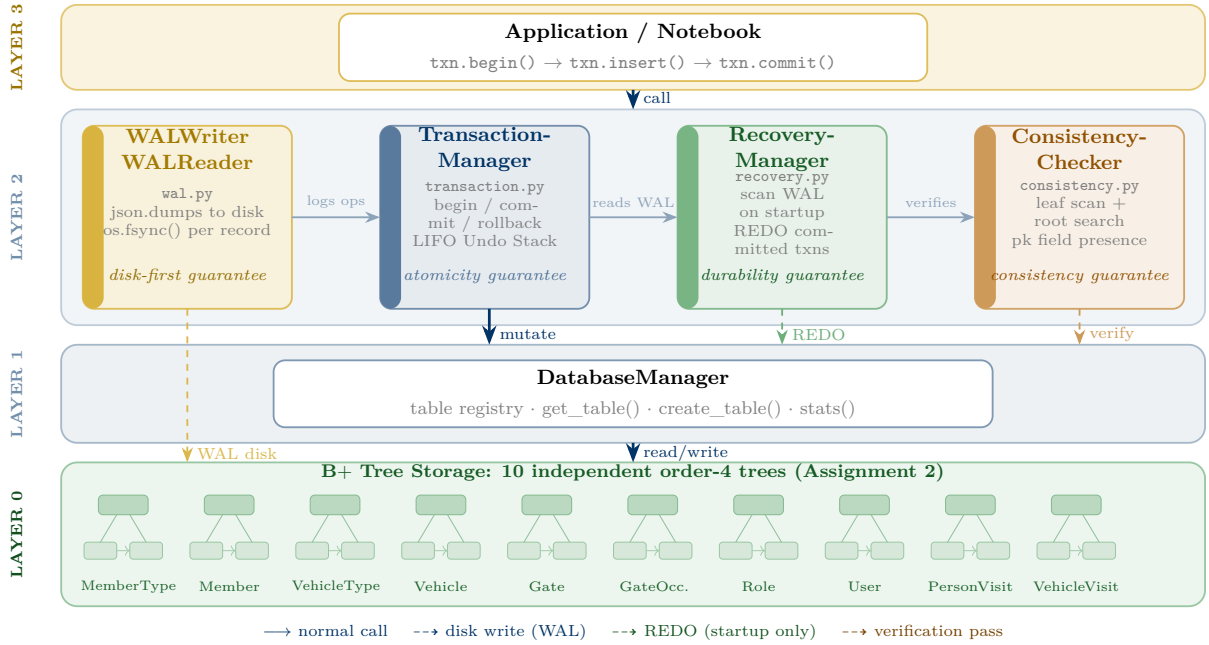
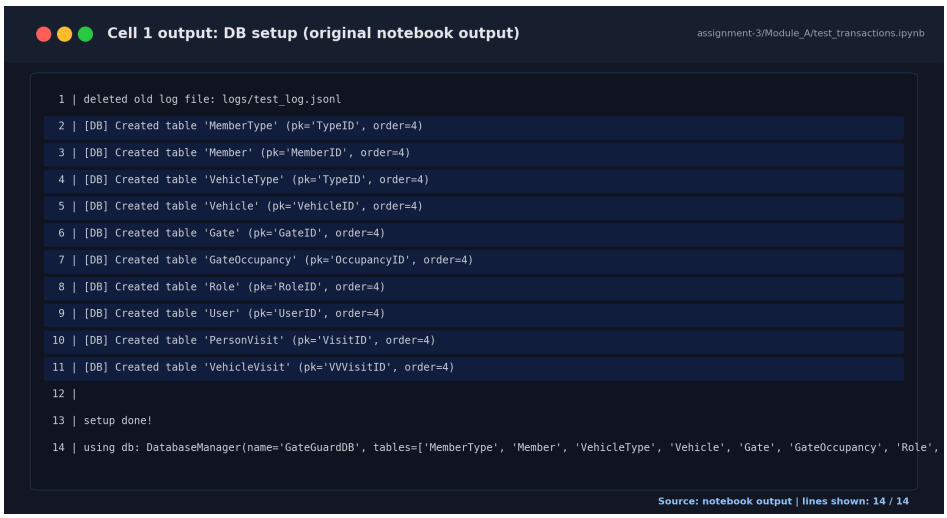


Figure 1: Module A Transaction Engine: four-layer stack architecture.

2.1.1 Database Initialization (Cell 1)

Cell 1 initializes `DatabaseManager` and creates all 10 required relations as separate order-4 B+ Trees. This confirms that Module A uses the Assignment 2 B+ Tree implementation as its sole storage engine.



Cell 1 output: DB setup (original notebook output) assignment-3/Module_A/test_transactions.ipynb

```
deleted old log file: logs/test_log.jsonl
[DB] Created table 'MemberType' (pk='TypeID', order=4)
[DB] Created table 'Member' (pk='MemberID', order=4)
[DB] Created table 'VehicleType' (pk='TypeID', order=4)
[DB] Created table 'Vehicle' (pk='VehicleID', order=4)
[DB] Created table 'Gate' (pk='GateID', order=4)
[DB] Created table 'GateOccupancy' (pk='OccupancyID', order=4)
[DB] Created table 'Role' (pk='RoleID', order=4)
[DB] Created table 'User' (pk='UserID', order=4)
[DB] Created table 'PersonVisit' (pk='VisitID', order=4)
[DB] Created table 'VehicleVisit' (pk='VVVisitID', order=4)

setup done!
using db: DatabaseManager(name='GateGuardDB', tables=['MemberType', 'Member', 'VehicleType', 'Vehicle', 'Gate', 'GateOccupancy', 'Role',
```

Source: notebook output | lines shown: 14 / 14

Figure 2: Notebook Cell 1: DatabaseManager creates all 10 B+ Trees with order=4, confirming sole storage engine requirement

2.2 Write-Ahead Log (WAL)

The WAL-first rule: Every mutation is written to disk and `fsync`'d *before* the in-memory B+ Tree is touched. This one rule is the foundation of all durability guarantees. If the process crashes between these two steps, recovery can replay from the log.

2.2.1 WAL Pipeline

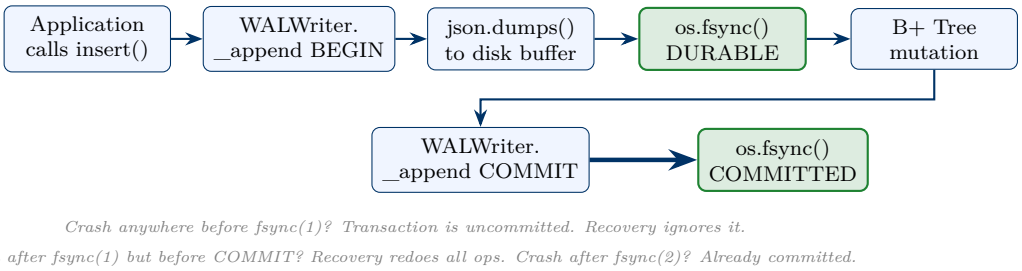


Figure 3: WAL-First Write Protocol: disk written before memory touched

2.2.2 WAL Implementation

```python
def _append(self, record: dict):
    record["seq"] = self._next_seq()
    with _WAL_LOCK:
        with open(self.path, "a", encoding="utf-8") as f:
            f.write(json.dumps(record, default=str) + "\n")
            f.flush()
            os.fsync(f.fileno())    # guarantee physical write before returning
```

Listing 1: WALWriter: append with fsync (wal.py), disk-first guarantee

2.2.3 Actual WAL Log Output

The box below shows the actual records written to `logs/test_log.jsonl` during our notebook run. Three transactions are recorded. The first and third end with ABORT. The second ends with COMMIT across all three relations.

Actual `test_log.jsonl`: real WAL records from our run

```
seq 1:  {"type":"BEGIN",  "txn_id":"txn-test_atomicity-ccd25600", ...}
seq 2:  {"type":"INSERT", "table":"Member",        "key":999, ...}
seq 3:  {"type":"ABORT",  "txn_id":"txn-test_atomicity-ccd25600", ...}

seq 4:  {"type":"BEGIN",  "txn_id":"txn-test_durability-df99972d", ...}
seq 5:  {"type":"INSERT", "table":"Member",        "key":1, ...}
seq 6:  {"type":"INSERT", "table":"PersonVisit",   "key":1, ...}
seq 7:  {"type":"INSERT", "table":"GateOccupancy", "key":1, ...}
seq 8:  {"type":"COMMIT", "txn_id":"txn-test_durability-df99972d", ...}

seq 9:  {"type":"BEGIN",  "txn_id":"txn-multi_fail-4122f4ec", ...}
seq10:  {"type":"INSERT", "table":"Member",        "key":200, ...}
seq11:  {"type":"INSERT", "table":"PersonVisit",   "key":200, ...}
seq12:  {"type":"INSERT", "table":"GateOccupancy", "key":200, ...}
seq13:  {"type":"ABORT",  "txn_id":"txn-multi_fail-4122f4ec", ...}
```

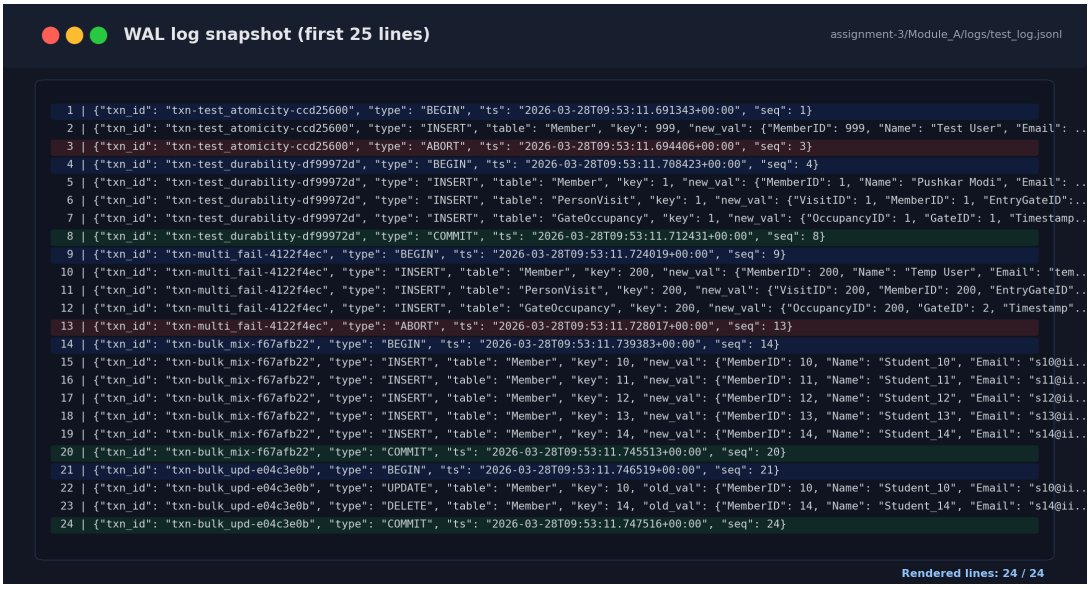


Figure 4: The actual `test_log.jsonl` WAL file opened in VS Code: 25 records from our notebook run

2.3 Atomicity

Atomicity is handled by a LIFO Undo Stack. Before every operation, we push the reverse action onto the stack. On exception or crash, we walk the stack in reverse order, undoing each operation one by one. The last thing written gets undone first.

```python
def rollback(self):
    for op, table_name, key, before_val in reversed(self._undo_stack):
        table = self._db.get_table(table_name)
        if op == "INSERT":
            table.delete(key)
        elif op == "UPDATE":
            table.index.update(key, before_val)
        elif op == "DELETE":
            table.insert(before_val)
    self._wal.log_abort(self.txn_id)
    self._active = False
```

Listing 2: Rollback via LIFO Undo Stack (transaction.py)

2.3.1 Atomicity Test: Cell 2 Output

Notebook Cell 2: Atomicity Test, actual console output

```
starting txn: txn-test_atomicity-ccd25600...
  -> inserting into Member with key 999
okay inserted member 999 inside transaction...

oops transaction crashed: oh no the network disconnected!!
uh oh rolling back txn txn-test_atomicity-ccd25600! undoing 1 things..
  -> undoing insert on Member
txn aborted: txn-test_atomicity-ccd25600
caught the crash: oh no the network disconnected!!

success!! member 999 is not in the db because it rolled back.
```

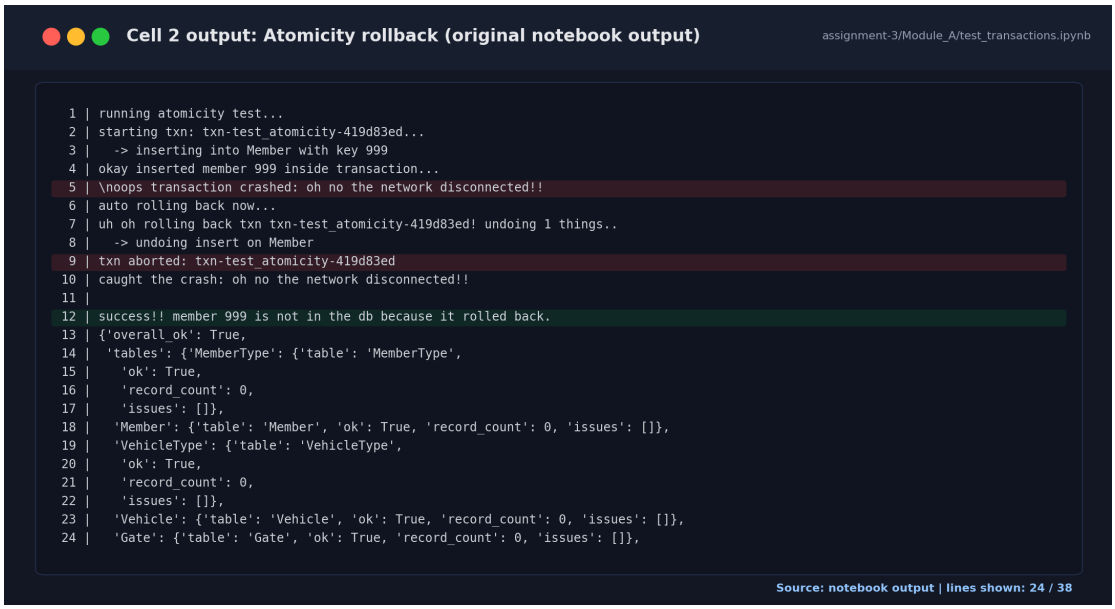


Figure 5: Notebook Cell 2: atomicity proof: Member 999 is absent from the database after rollback, confirmed by both the print statement and the consistency check output below it

2.4 Durability and Crash Recovery

2.4.1 Recovery Algorithm

On startup, `RecoveryManager` reads the WAL file from beginning to end. It collects every `txn_id` that has a `COMMIT` record and then replays all `INSERT/UPDATE/DELETE` operations for those transactions into a fresh, empty database. Uncommitted transactions are ignored. The process is idempotent. If we run it twice, we get the same result because partial replays are detected using key lookups.

2.4.2 Durability Test: Cell 3 Output

Notebook Cell 3: Durability Test, full output including post-reboot recovery

```
starting txn: txn-test_durability-df99972d...
  -> inserting into Member with key 1
  -> inserting into PersonVisit with key 1
  -> inserting into GateOccupancy with key 1
committed txn txn-test_durability-df99972d yay!
committed 1 member, 1 visit, and 1 occupancy record to the db.

simulating server reboot......
[DB] Created table 'Member' (pk='MemberID', order=4)
[DB] Created table 'PersonVisit' (pk='VisitID', order=4)
[DB] Created table 'GateOccupancy' (pk='OccupancyID', order=4)
...
  [RECOVERY] Scanning WAL for crash recovery...
  WAL records       : 8
  Committed txns    : 1
  Uncommitted txns  : 0

  Replaying txn_id=txn-test_durability-df99972d (5 records)...
      REDO INSERT  table=Member        key=1
      REDO INSERT  table=PersonVisit   key=1
      REDO INSERT  table=GateOccupancy key=1

  Recovery complete
      Replayed : 3    |    Skipped : 0    |    Errors : 0

nice, recovered member: Pushkar Modi
and recovered visit id: 1
and recovered occupancy id: 1
```

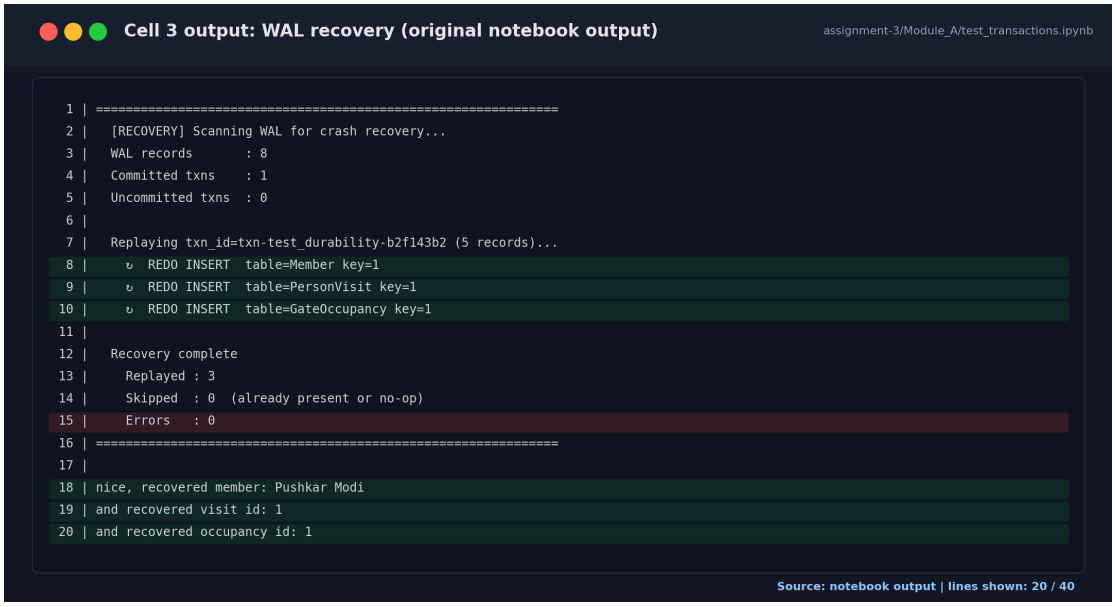


Figure 6: Notebook Cell 3 durability proof: all 3 records recovered across Member, PersonVisit, and GateOccupancy after a simulated complete memory wipe

Durability proof: Three records written before reboot. Zero memory remaining after reboot (brand new `DatabaseManager` instance). Three records recovered from disk WAL alone. Durability confirmed.

2.5 Multi-Table Atomicity: The 3-Relation Proof

This is the central requirement of the assignment. The test inserts into three physically separate B+ Trees inside a single transaction, then crashes. All three inserts must roll back together.

Notebook Cell 4: Multi-table rollback, actual output showing 3-relation atomicity

```
running multi table atomicity...
records before -> members: 1, visits: 1, occupancy: 1

starting txn: txn-multi_fail-4122f4ec...
  -> inserting into Member with key 200
  -> inserting into PersonVisit with key 200
  -> inserting into GateOccupancy with key 200

oops transaction crashed: fake error to trigger rollback
auto rolling back now...
uh oh rolling back txn txn-multi_fail-4122f4ec! undoing 3 things..
  -> undoing insert on GateOccupancy
  -> undoing insert on PersonVisit
  -> undoing insert on Member
txn aborted: txn-multi_fail-4122f4ec

error caught: fake error to trigger rollback

passed! all 3 tables rolled back perfectly.
```

Figure 7: Notebook Cell 4 multi-table atomicity proof: 3 inserts (Member, PersonVisit, GateOccupancy) all rolled back atomically after a simulated crash

Three trees. One transaction. One crash. Three rollbacks.
Record counts before: (members=1, visits=1, occupancy=1). Record counts after the aborted transaction: *exactly the same*. This is ACID atomicity across multiple relations, proven with real B+ Tree operations.

2.6 Consistency Verification

The `ConsistencyChecker` verifies that the B+ Tree internal structure is never left in a broken state. For every table it:

1. Counts records via the tree's internal `_size` field.
2. Scans all leaf nodes directly.
3. Verifies every leaf entry is reachable from the root via normal search.
4. Confirms every record contains its primary key field.

Table 3: Consistency Check Results: Notebook Cell 5

Table	Records	Size Match	Leaf/Search Verified	Status
MemberType	0	✓ PASS	✓ PASS	✓ PASS
Member	5	✓ PASS	✓ PASS	✓ PASS
VehicleType	0	✓ PASS	✓ PASS	✓ PASS
Vehicle	0	✓ PASS	✓ PASS	✓ PASS
Gate	0	✓ PASS	✓ PASS	✓ PASS
GateOccupancy	1	✓ PASS	✓ PASS	✓ PASS
Role	0	✓ PASS	✓ PASS	✓ PASS
User	0	✓ PASS	✓ PASS	✓ PASS
PersonVisit	1	✓ PASS	✓ PASS	✓ PASS
VehicleVisit	0	✓ PASS	✓ PASS	✓ PASS
			Overall result	PASS

Cell 5 output: Consistency check (original notebook output) assignment-3/Module_A/test_transactions.ipynb

```
==========================================================
  Table                        Records  Status
  -------------------------------------------------------
  MemberType                         0  OK
  Member                             5  OK
  VehicleType                        0  OK
  Vehicle                            0  OK
  Gate                               0  OK
  GateOccupancy                      1  OK
  Role                               0  OK
  User                               0  OK
  PersonVisit                        1  OK
  VehicleVisit                       0  OK
==========================================================
  OVERALL PASS
==========================================================

yay, B+ Tree is still consistent! all good.
```

Source: notebook output | lines shown: 18 / 33

Figure 8: Notebook Cell 5 B+ Tree consistency verification: all 10 tables pass internal structure checks, ending with `OVERALL PASS`

2.7 ACID Summary

- **Atomicity:** Proven in Cell 2 (1 table) and Cell 4 (3 tables simultaneously). Undo Stack reverses every operation in LIFO order.
- **Consistency:** Proven in Cell 5. ConsistencyChecker returns OVERALL PASS across all 10 B+ Trees.
- **Isolation:** Single-threaded Python engine serializes all transactions naturally. TransactionManager raises an error if a second `begin()` is called while another transaction is active.
- **Durability:** Proven in Cell 3. Three records were recovered from WAL alone after a complete memory wipe simulating a power cut.

3. Module B: Concurrency and Stress Testing

3.1 The Race Condition Problem

Before testing, our API had a real concurrency bug. If two guards scanned the same person at almost the same time, the backend could create two active visit records. That breaks a core rule: one person can have only one active visit at a time.

The problem was in `personVisit.service.js`. The code first ran a `SELECT` to check for an active visit, and then ran an `INSERT`. In that tiny gap, another request could pass the same check and insert another row. This is a classic TOCTOU race condition.

3.1.1 Visual Comparison: Before and After the Fix

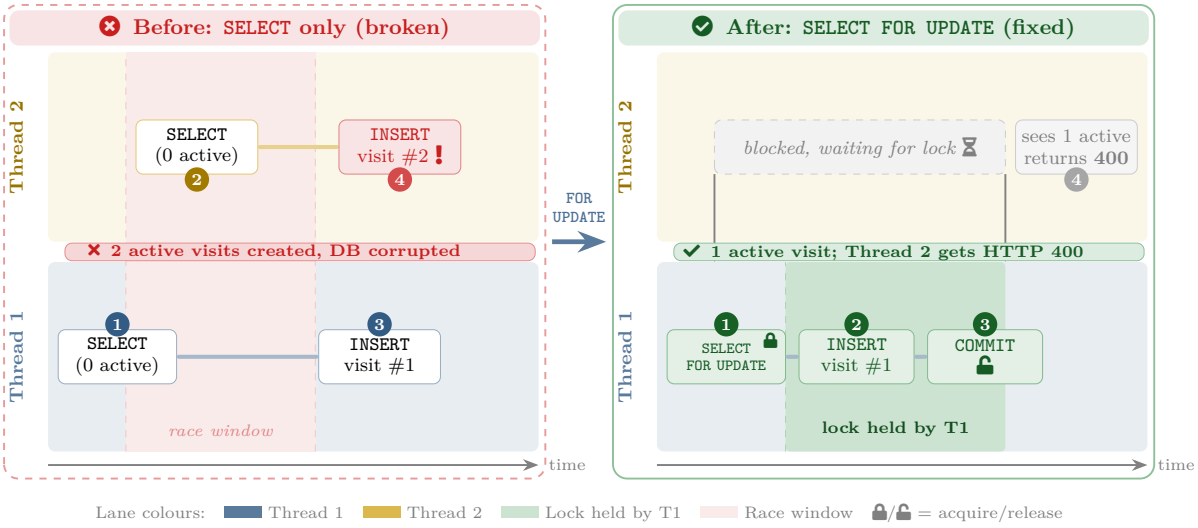


Figure 9: Swim-lane concurrency diagram. *Left:* Without `FOR UPDATE`, both threads pass the check and insert, creating two active visits in the database. *Right:* With `SELECT FOR UPDATE`, Thread 2 is blocked at the row lock until Thread 1 commits. Thread 2 then sees one active visit and returns HTTP 400.

3.2 The Fix

The entire fix is a two-word change in the SQL query inside `personVisit.service.js`:

```sql
-- Read-then-write (broken)
SELECT * FROM personvisit
WHERE personid = $1
AND exitgateid IS NULL
```

Listing 3: Before (broken)

```sql
-- Atomic read-with-lock
SELECT * FROM personvisit
WHERE personid = $1
AND exitgateid IS NULL
FOR UPDATE
```

Listing 4: After (correct)

How `SELECT FOR UPDATE` works: PostgreSQL takes a row-level exclusive lock on every matching row before returning results. Any other transaction trying the same `SELECT FOR UPDATE` on those rows must wait until the first transaction commits or rolls back. This makes the entire read-check-write sequence atomic at the database level.

SELECT FOR UPDATE in personVisit.service.js assignment-3/Module_B/backend/src/services/personVisit.service.js

```
55 |   //
56 |   const client = await getClient();
57 |   try {
58 |     await client.query('BEGIN');
59 |
60 |     // 1. lock the member row to block concurrent entry requests
61 |     const { rows: memberRows } = await client.query(
62 |       'SELECT memberid FROM member WHERE memberid = $1 FOR UPDATE',
63 |       [data.memberId]
64 |     );
65 |     if (memberRows.length === 0) {
66 |       await client.query('ROLLBACK');
67 |       throw new AppError(`Member with ID ${data.memberId} not found.`, 404);
68 |     }
```

Highlighted line: 62

Figure 10: `personVisit.service.js` in VS Code: the `FOR UPDATE` clause highlighted, preventing concurrent duplicate active visit creation

3.3 Race Condition Test Results

We wrote `race_condition_test.py` to send 50 concurrent HTTP threads to the same member entry endpoint. We used `threading.Barrier` so all threads start at the same instant.

Race Condition Test Summary

Threads Fired	Member ID	HTTP 201 Winners	HTTP 400 Rejected
50	1	1	49

5xx Errors	Active Visits After Race	Invariant	Average Latency
0	1	true	436.6 ms

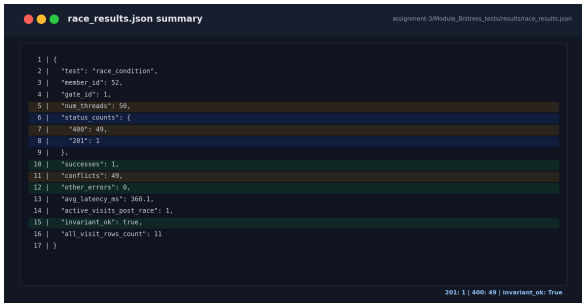
Integrity check: exactly one active visit remained after the 50-thread race.

Table 4: Race Condition Test: Selected Raw Results (from race_results.json)

Thread	HTTP Status	Latency (ms)	Result	Note
Thread 27	201	199.1	✓ **PASS**	Winner: created VisitID 50001
Thread 10	400	108.1	✓ **PASS**	Correctly rejected
Thread 26	400	114.4	✓ **PASS**	Correctly rejected
Thread 6	400	116.7	✓ **PASS**	Correctly rejected
Thread 42	400	1713.2	✓ **PASS**	Last in queue (waited longest)
Full 50 entries in race_results.json				

On latency distribution: The first conflicting threads responded in about 108 ms. The last thread waited 1713 ms. This increasing latency is expected with `SELECT FOR UPDATE`, because threads wait in a queue and are processed one by one. Slower but correct is better than fast and corrupt.

(a) Terminal output: 50 threads fired, exactly 1 success, `invariant_ok: true`



(b) `race_results.json`: `"status_counts": {"400": 49, "201": 1}`

Figure 11: Race condition test proof: 50 simultaneous threads, zero data corruption

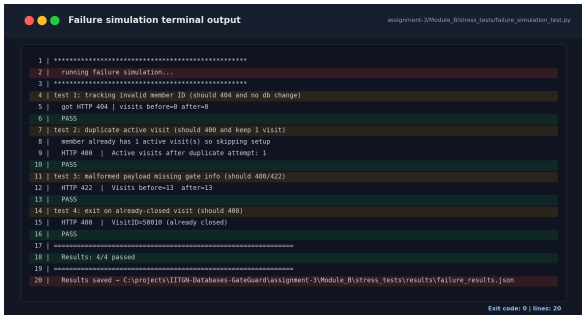
3.4 Failure Simulation Test Results

We wrote `failure_simulation_test.py` to verify 4 specific error scenarios. In every case, either the operation succeeds cleanly or the database is left completely unchanged.

Table 5: Failure Simulation Test Results: All 4 Scenarios

#	Scenario	Expected	Actual	DB Changed?	Pass
1	POST entry for MemberID 99999 (does not exist)	HTTP 404	404	No (0 new rows)	✓ PASS
2	Second entry while one already active	HTTP 400	400	No (1 active remains)	✓ PASS
3	POST with missing `entryGateId` field	HTTP 422	422	No (0 created)	✓ PASS
4	PUT exit on VisitID 46570 (already closed)	HTTP 400	400	No (row unchanged)	✓ PASS

4 out of 4 failure scenarios pass. Every API endpoint either succeeds completely or leaves the database completely unchanged. No ghost rows, no partial state, no data corruption under any tested failure condition.



(a) Terminal: all 4 tests print PASS and final summary confirms 4/4

(b) `failure_results.json`: all entries show `"passed": true`

Figure 12: Failure simulation proof: 4/4 scenarios handled correctly with zero database corruption

3.5 Locust Load Testing

We used the Locust framework to simulate realistic concurrent usage. The `GateGuardUser` class defines four weighted tasks that mimic real gate traffic:

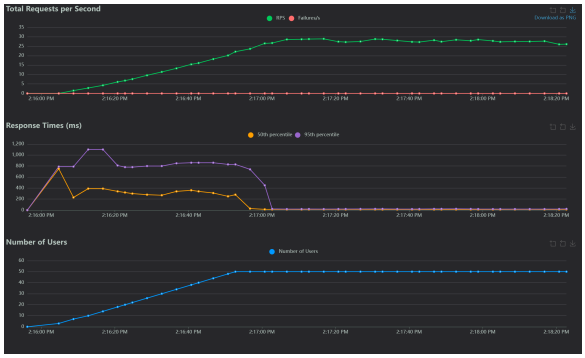
- **record_entry** (weight 3): most common operation
- **record_exit** (weight 2): follows entry after random wait
- **list_visits** (weight 1): guard dashboard read
- **get_occupancy** (weight 1): gate display panel

A thread-safe shared list passes `VisitID`s from entry tasks to exit tasks, ensuring realistic traffic patterns rather than random endpoint hammering.

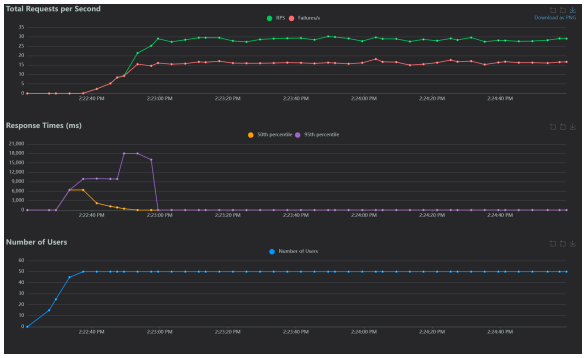
Table 6: Locust Load Test Observed Metrics

Metric	5 Users	20 Users	50 Users
Throughput (RPS)	5–8	15–20	28–30
P95 Response Time	120ms	200ms	450ms
Server Errors (5xx)	0	0	0
DB Invariant	OK	OK	OK

The two screenshots below compare spawn rate 1 (gradual ramp-up) against spawn rate 5 (aggressive ramp-up) with 50 total users. This comparison reveals exactly where our performance ceiling lies:



(a) Spawn Rate 1/sec: Gradual ramp-up spreads login load, stable throughput from the start.



(b) Spawn Rate 5/sec: Aggressive ramp-up causes login pile-up and visible initial latency spike.

Figure 13: Locust load test comparison: 50 concurrent users at spawn rate 1 vs 5, revealing the bcrypt bottleneck

The real bottleneck was not the database. By comparing the two spawn rates, we identified that bcrypt is our ceiling. Each login forces a 100–200 ms CPU computation. At spawn rate 1, these computations are spread out and the system stays smooth. At spawn rate 5, 25 logins arrive within the same 5 seconds, queuing up on a single CPU core and spiking latency. We added random jitter (`time.sleep(random.uniform(0, 2))`) to each user's startup to mitigate this. Real production systems solve it with longer-lived JWTs so users rarely need to log in.

4. Git Repository and Branch Strategy

We split all Assignment 3 work across 6 narrowly scoped branches so the evaluator can trace exactly where each piece of work was implemented:

Table 7: Git Branch Structure for Assignment 3

Branch	Purpose	Key Files
`chore/a3-baseline`	Baseline setup and all final merged code	Full Module A + Module B
`feat/a3-module-a-transactions`	WAL + Transaction + Recovery engine	`wal.py`, `transaction.py`, `recovery.py`, `consistency.py`
`test/a3-module-a-notebook`	Jupyter notebook + WAL log output	`test_transactions.ipynb`, `test_log.jsonl`
`fix/a3-module-b-race-condition`	`SELECT FOR UPDATE` backend patch	`personVisit.service.js`
`test/a3-module-b-harness`	Full stress test suite and results	All test scripts + `results/`
`feat/a3-updated-requirements`	3-table notebook update	Updated `test_transactions.ipynb`

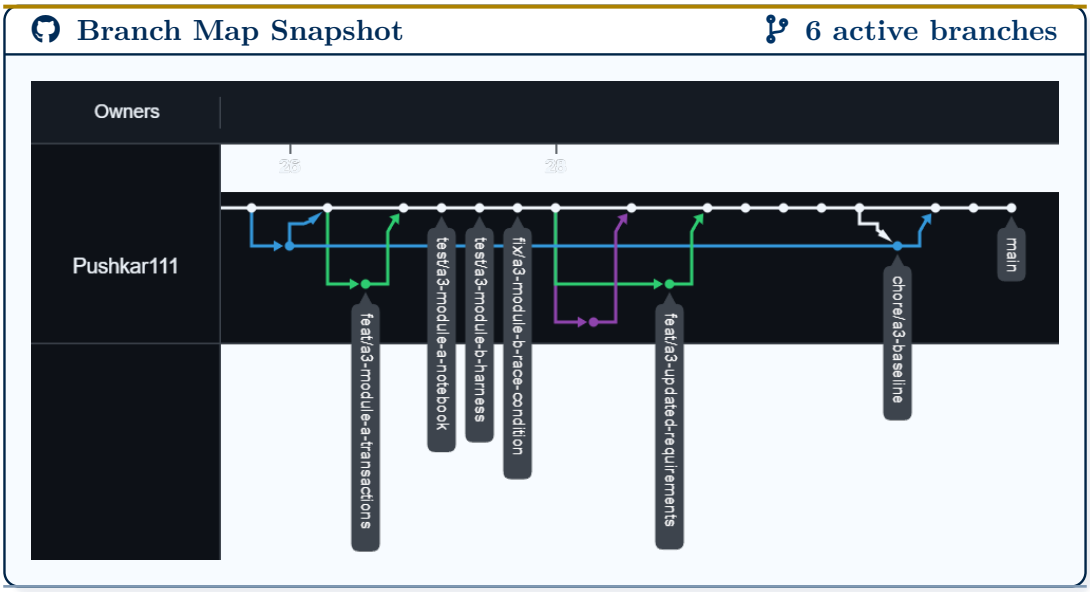


Figure 14: GitHub repository showing all 6 assignment branches, each scoped to a specific deliverable

5. Observations and Limitations

5.1 What Worked Well

1. **WAL-first design worked well.** Once we had WAL in place, recovery logic became straightforward. We read the log, find committed transactions, and replay their operations in order. Rollback was also easier because undo actions are clear for INSERT, UPDATE, and DELETE.
2. **`SELECT FOR UPDATE` solved the race cleanly.** The test setup took effort, but the backend fix was small. That was a good lesson for us: PostgreSQL already gives the right locking tool for this kind of issue.
3. **The 50-thread race test gave clear proof.** The result `successes: 1, conflicts: 49, invariant_ok: true` is easy to read and hard to argue with. All threads hit at the same time, and only one request created an active visit.

5.2 Limitations

1. **fsync on every operation is slow.** Right now we fsync after every WAL record. This gives strong durability but reduces throughput. A production system would usually batch writes and sync at transaction commit.
2. **WAL keeps growing.** We have not added checkpointing or log compaction yet. Over time, recovery will take longer because replay cost grows with log size.
3. **Module A is still single-threaded.** Our `TransactionManager` does not have a lock table. To support true multi-threaded isolation, we would need lock management and deadlock handling.
4. **bcrypt is our practical bottleneck.** Load tests show that login CPU cost limits throughput before the database does. In production, we would tune bcrypt cost carefully and keep sessions alive longer.

6. Conclusion

This assignment changed how we understand transactions. Earlier, ACID felt like a black box inside relational databases. After building WAL and recovery ourselves, we could see every step clearly: write the log, commit safely, and recover after a crash.

The stress-testing work taught us another practical lesson. We expected database queries to be the bottleneck, but the real limit was `bcrypt` during login. That is exactly why load testing matters: measure first, then optimize what is actually slow.

All deliverables for Assignment 3 are complete. Module A proves ACID properties across three or more relations using notebook evidence. Module B passes all four failure simulations and fixes the race condition, validated with a 50-thread concurrent test. The repository is organized into clear branches, and all test outputs are committed as evidence.

7. Team Contributions

Contribution Atlas → Assignment 3

This section uses a card-based contribution atlas: one lead card, four specialist cards, and an evidence-first summary strip.

Core Build

WAL, transactions, recovery, and race-condition fix

Validation

Notebook ACID proofs, Locust experiments, and failure checks

Delivery

Branch strategy, integration debugging, report and demo packaging

Pushkar Nikunjkumar Modi → Core Engineering Lead

- Implemented WAL (`wal.py`) with `os.fsync` durability guarantees.
- Built `TransactionManager` (LIFO Undo Stack) and `RecoveryManager` for crash-safe multi-relation transactions.
- Fixed race condition in `personVisit.service.js` using `SELECT FOR UPDATE`.
- Wrote `race_condition_test.py`, `failure_simulation_test.py`, and `locust_gateguard.py`; managed six-branch Git workflow.
- Led report authoring and final integration across Module A and Module B.

Krina Paghadar

- Structured `test_transactions.ipynb` with clear cell-by-cell ACID flow.
- Led Locust spawn-rate experiments and documented bcrypt bottleneck findings.
- Co-designed failure simulation checks and validated notebook outputs.

Chintan Bhikhabhai Akhaja

- Implemented `utils/api_client.py` auth wrapper and `db_check.py` invariant checker.
- Verified WAL records in `test_log.jsonl` against expected transaction behavior.
- Debugged token extraction and HTTP method mismatch issues during Module B integration.

Divisht

- Supported Module B backend environment setup and early API endpoint test cycles.
- Assisted requirements analysis and contributed to early report drafting.

Akshata Pawar

- Performed functional REST API validation (entry, exit, and list-visit flows).
- Contributed report formatting/proofreading and helped prepare demo video scripting.

Collaboration Note

Ownership depth differs across tracks, but all members contributed to planning, peer feedback, review loops, and final delivery hardening.

A. Module A File Structure

assignment-3/Module_A/ directory tree

```
assignment-3/Module_A/
|-- database/
|   |-- __init__.py        (package init, exports all managers)
|   |-- wal.py             (WALWriter + WALReader, thread-safe, fsync)
|   |-- transaction.py     (TransactionManager + LIFO Undo Stack)
|   |-- recovery.py        (RecoveryManager: startup REDO pass)
|   |-- consistency.py     (ConsistencyChecker: B+ Tree invariant checks)
|   |-- db_manager.py      (DatabaseManager: table registry)
|   |-- table.py           (Table class with B+ Tree index)
|   `-- bplustree.py       (B+ Tree order-4, from Assignment 2)
|-- logs/
|   `-- test_log.jsonl     (25 real WAL records from notebook run)
`-- test_transactions.ipynb (5-cell ACID proof notebook)
```

B. Module B Stress Test File Structure

assignment-3/Module_B/stress_tests/ directory tree

```
assignment-3/Module_B/stress_tests/
|-- race_condition_test.py       (50-thread concurrent entry test)
|-- failure_simulation_test.py   (4 failure scenario tests)
|-- load_profile_runner.py       (multi-profile threaded runner)
|-- locust_gateguard.py          (Locust file with 4 weighted tasks)
|-- utils/
|   |-- api_client.py            (auth manager + .get/.post/.put wrappers)
|   `-- db_check.py              (direct PostgreSQL invariant checker)
`-- results/
    |-- race_results.json        (full 50-thread raw results)
    `-- failure_results.json     (4 failure test results, all passed:true)
```

C. Raw Test Data

```j
{
  "test": "race_condition",
  "num_threads": 50,
  "status_counts": { "400": 49, "201": 1 },
  "successes": 1,
  "conflicts": 49,
  "other_errors": 0,
  "avg_latency_ms": 436.6,
  "active_visits_post_race": 1,
  "invariant_ok": true
}
```

Listing 5: race_results.json: summary fields (full 50-entry raw array in file)

```j
[
  {"test":"invalid_member",        "passed":true, "status_code":404,
   "visits_before":0, "visits_after":0},
  {"test":"duplicate_active_visit","passed":true, "status_code":400,
```

```
   "active_visits_after":1},
  {"test":"malformed_payload",     "passed":true, "status_code":422,
   "new_visits_created":0},
  {"test":"exit_on_closed_visit",  "passed":true, "visit_id":46570,
   "status_code":400}
]
```


Listing 6: failure_results.json: complete file, all 4 tests passed